UNIVERSITY OF COPENHAGEN
FACULTY OF SCIENCE

Master's Thesis

Eliminating Timing Channels in Microkernels

Integrating Time Protection Mechanisms into FreeRTOS

Simon Vinding Brodersen(rzn875)

Advisor: Thomas Philip Jensen

Submitted: May 29, 2026

Abstract

Timing channels present a significant security threat in modern computing, allowing unauthorized information flow through observable variations in execution time. Research has been done on creating secure microkernels designed to mitigate these channels from the ground up, while other research focuses on closing timing channels at the user level. It remains an open challenge to gather these findings for a more general and mainstream OS, and this thesis aims to investigate the feasibility of integrating time protection mechanisms into the widely used FreeRTOS kernel.

FreeRTOS provides a default priority-based preemptive scheduler, which is inherently vulnerable to timing channels. To address this, the project implements a domain-level scheduler that partitions execution into fixed time slices, ensuring that the execution of tasks in one domain does not affect the timing observed by another. Key to this implementation is the temporal fence (fence.t) instruction during domain switches to clear shared microarchitectural states. Further, constant-time domain creation mechanisms are introduced, allowing dynamic resource reconfiguration without introducing new timing channels.

The implementation was evaluated using QEMU system emulation. The results demonstrate that the modified kernel achieves constant-time domain scheduling with negligible variance with regard to instruction counting. While evaluation on true hardware remains undone, this work provides a foundation towards bringing timing protection to mainstream embedded systems.

Contents

1 Introduction

Information channels are methods of communication in an unintended manner. One such example is timing channels, which occur when a task's execution time varies based on the execution of another task in the system. Whether inadvertently or purposely, the other task is given unintended information about the current task by altering its execution time. In this way, timing channels bypass the operating system's (OS's) security enforcement leaking information through timing of observable events. At first many attacks were targeting different cryptographic libraries and leaking information about secret keys through these timing channels [1], [2]. Further, attacks have shown that it is possible to gather information across VMs running in a cloud computing setting [3]. Not only that, side-effects from out-of-order executions on modern processors gave way to the Meltdown attacks to read arbitrary kernel-memory locations including personal data and passwords [4]. Different mitigations are moving forward, such as hiding kernel memory from user space tasks [5] to prevent Meltdown attacks, and a move to more constant time programming in cryptographic libraries with help from domain specific languages(DSL) such as FaCT [6] that aims to guarantee constant-time execution with respect to secretly marked information. While these mitigations do prove valuable, more modern CPUs' speculative execution still leaves doors open for more sophisticated attacks such as the ones seen in [7].

The attack front is clear, but the solution is not as simple. While some proposed solutions focus on the user side implementing secure programs, this creates a huge burden on the developer to implement secure code. Instead implementations of secure microkernels [8], [9] attempt to guarantee security through the OS. The operating system would be in charge of partitioning domains, such that no channel between domains is possible. This is already done for memory, and examples of formally proven memory-safe kernels are available [10]. Attempts to create similar proof against the presence of timing channels have been carried out [11], but as far as this paper is aware no kernel is formally verified against timing channels.

Many of these attempts are focused on the creation of a highly secure OS from the ground up, providing timing protection as a foundation of the OS. While this is a more robust solution for a truly safe OS the authors of [9] hint that "there is no reason time protection cannot be implemented in other systems...", something which could bring timing protection to more mainstream applications.

FreeRTOS is an example of such a mainstream system, which provides a default priority-based preemptive scheduler, which is inherently vulnerable to timing channels. While ground-up microkernels have proven effective against timing channels, adding these time protection mechanisms into existing operating systems remains an open

issue. To investigate this, the thesis addresses the following research question: *Can time protection mechanisms be integrated into a mainstream OS to mitigate timing channels?*

To this end, this thesis investigates a defence mechanism composed of a domain-level scheduler with deterministic flushing on domain switching via the temporal fence instruction. This approach focuses primarily on mitigating scheduling-based timing channels and leaks through the shared microarchitectural state, both of which are discussed further in Section 2.3.

To achieve this mechanism, this thesis adds the following contributions.

- A domain-level scheduler that partitions execution into fixed time slices, ensuring execution of tasks in one domain does not affect the schedulability of a task in another domain.
- Temporal isolation with the use of temporal fence instruction during domain switches to clear shared microarchitectural state.
- An evaluation of the domain-level scheduler using QEMU system emulation with instruction-based time counting, showing constant-time domain scheduling.

The remainder of this thesis is organized as follows. Section 2 provides the technical background on timing channels, the `fence.t` instruction, and the FreeRTOS kernel. Section 3 surveys related work on user-level mitigations, the seL4 time-protection kernel, and the S3K real-time operating system. Section 4 analyses the FreeRTOS scheduler's vulnerability to timing channels and explores different methods for closing them, and introduces the domain-level scheduler. Section 5 details the implementation of this scheduler within the FreeRTOS-MPU RISC-V port. Section 6 presents the evaluation using QEMU system emulation with two workloads and shows a constant round trip using the cycle-count. Section 7 outlines directions for future work, and Section 8 discusses the broader implications of partitioned hardware and time protection. Finally, Section 9 concludes the thesis.

2 Background

To address the challenges of time protection, it is essential to first establish a general understanding of how these vulnerabilities manifest and key terms used when explaining timing channels. This section provides the technical foundation for the work that follows and aims to build a foundational understanding about what channels are and how isolation can be used as a means for protection.

The remainder of this section is structured as follows. First, Section 2.1 explores side-channels, detailing how tasks can involuntarily leak information through execution time and speculative execution. Section 2.2 investigates the extension of this, where two colluding processes intentionally attempt to bypass security boundaries. Section 2.3 categorizes the specific timing channels an operating system faces, splitting them into scheduler-based, shared-state and interrupt-based vulnerabilities. Section 2.4 shifts the focus toward mitigation, detailing the principles of spatial and temporal isolation, before Section 2.5 reviews the hardware-assisted `fence.t` instruction designed to eliminate microarchitectural state leakage with temporal isolation. Finally, Section 2.6 introduces the FreeRTOS ecosystem and the RISC-V platform used as the foundational environment for this thesis.

2.1 Side-channels

Side-channels are unintended information channels that leak information about the process. These channels infer secret data often through observable execution time of a process. The general case of such a channel can be seen in the snippet below:

```
1: if secret then
2:     Some long computation
3: else
4:     Do nothing
5: end if
```

This algorithm is a naive example, which simply branches on some secret, and the execution time of the program reveals the secret. If for example this algorithm was part of some library, then an attacker could infer information about the secret by simply measuring the execution time of the call to the algorithm. This example is quite naive, and is one of the side-channels constant time programming at the user level can prevent, such that one might do an equally long computation in both branches of the if statement. However, this still comes with some precautions, because even though the original program might seem sensibly constant time, some compilers might try to optimise the code, which could introduce new side-channel attacks such as the one seen in [12].

Further, more complex examples of side-channels includes those specified within [7], where speculative execution can be used to leak information about an otherwise safe algorithm. An example from the paper is the following:

This function could again be part of some library or system call, which receives an unsigned integer x from the user. The process, which runs this library code or

```
1: if x < array1_size then
2:     y = array2[array1[x] * 4096]
3: end if
```

system call has access to both array1 and array2. It does a bounds check on x, to ensure there is no out of bounds access, which could trigger an exception or access sensitive memory. If this algorithm was called multiple times, with correct input, i.e. where $x < array1\_size$, then an attacker could train the branch predictor on modern CPUs making the CPU speculatively execute the expected path, which could bring in the location read from `array2` into the cache, and the attacker could then use attacks such as Flush+Reload or Prime+Probe to reveal which part of array2 was loaded into the cache. With this, it can gather the index of the loaded item, and can figure out what was read from array1[x], which could be some arbitrary secret that was obtained by the out of bounds indexing that was made possible by the branch predictor. A more detailed explanation along with a proof-of-concept is provided in [7], but this small example shows how seemingly safe code, might still leak secret information.

A multitude of such attacks, which make use of these side-channels are presented in [13], [14].

2.2 Covert channels

While side-channels are passive and exploit unintended leakage, covert channels involve two processes working together. This usually revolves around a sender (Trojan) and a receiver (spy), where the sender has access to high security data but is restricted in its communication with the receiver. To bypass this, the sender and receiver collude by any means possible to share information. This could be via the same techniques as the side-channels, but it could also be any use of shared state between the two processes.

Generally two main categories of covert channels exist; **Storage channels:** which modify some stored object, e.g. a file or some metadata; **Timing channels:** which change the time of different observable events for the receiver. For this thesis storage channels are not referenced, as kernels such as seL4 have already formally proven the absence of them [10].

Covert timing channels work through the same mechanisms as the aforementioned side-channel, but pose as a worst case, where the sender can make use of any and all leakage methods to convey the information. This way, one is not limited to the visible gadgets that might exist in a vulnerable program, but the information might

also be conveyed through repeated use of non-constant time system calls that depend on some shared resource, or as the aforementioned example shared use of the cache.

2.3 Timing channels

Both side-channels and covert channels rely on some shared resource whose state or timing can be observed by a receiver. In the context of an operating system kernel, these shared resources take several distinct forms. This subsection presents some of the timing channels an OS kernel could face, categorising the items that enable the channel.

Scheduler-based channels The OS scheduler itself can function as an unintended information channel. The scheduler controls which tasks get to run and might inadvertently leak important information about the schedulability of a task through this behaviour. Examples of a side-channel in the priority based real time schedulers can be seen in [15], where they demonstrate how an unprivileged, low-priority task (in user space) can infer execution behaviours of critical, high-priority periodic tasks. They show how this information is useful to launch other more devastating attacks and demonstrate two such examples. Further, in [16] they study more general shared schedulers and how information leakage arise in the context of shared event schedulers and argue about the different trade-off between privacy and performance.

A simple example of a timing channel could be one present in a priority-based scheduler. In such a scheduler the highest priority can inhibit execution of lower priority tasks. This could be used as a covert channel, where the higher priority task can signal either a 0 or 1 by simply going to sleep, thus allowing the lower priority task to run, or it could block execution of the lower priority task. The lower priority task can measure its last wake time, and infer if the higher priority task attempted to signal either a 0 or 1.

Shared state channels Microarchitectural resources such as caches, TLBs, and branch predictors are typically shared across tasks executing on a core. As demonstrated in Section 2.1, the state left in these components by one task can be observed by another through timing measurements, creating a class of channels that is both broad and difficult to eliminate in software alone. These are the channels that often require explicit help from hardware mechanisms such as the temporal fence instruction [17], [18], which are investigated further in Section 2.5.

Interrupt-based channels Many shared resources in an OS are protected by locks or critical sections. The interrupt mechanism itself, as well as the latency it introduces, can carry timing information. A task could trigger different interrupts, which would delay another task leading to another timing channel. A task could also simply hold a lock and in doing so prevent another task from gathering the same lock, which again would delay execution and transfer information between the two tasks.

Mitigating these channels requires a combination of mechanisms to provide true time protection.

2.4 Isolation

Throughout many timing channel mitigations, isolation is a common factor. Generally there exist two types of isolation, namely **spatial** and **temporal** isolation. Spatial isolation partitions software into distinct memory spaces, which prevents separate domains from sharing the same underlying hardware. This separation is most commonly used for DRAM memory, where the OS kernel relies on hardware support like the memory management units (MMUs) or memory protection units (MPUs). The MMU translates virtual memory addresses into physical addresses in main memory and the OS can separate different processes usage entirely. It also generally provides memory protection by implementing access control rules and regulations stopping illegal usage of particular memory locations.

The latter isolation type, namely temporal isolation, must ensure that the service received from shared resources by the software in one domain cannot be affected by the software in another domain [19]. An example, which would require such isolation is any shared part of the OS between different domains. Calling shared OS code should not have any measurable timing effect on another domain calling the same OS code. The solution is often to reset the shared resource back to a known default before any partitioned item is run. This guarantees that each process runs from a clean state which is unaffected by previous use.

To achieve temporal isolation one must clean any microarchitectural state that could be affected by process usage. This is where an instruction such as `fence.t` [17] would come into play. The instruction clears any shared microarchitectural state that cannot be partitioned spatially allowing for a deterministic state for a new process to run in.

2.5 temporal_fence_t

Following the difficulties identified in [9], it was clear that hardware improvements are needed before time protection can be fully achieved. The paper [17] introduce one such hardware improvement. First, it demonstrates the presence of serious timing channels within an in-order RISC-V core and confirms that the current ISA lacks mechanisms to close them reliably.

Following this, they present two versions of the `fence.t` instruction: a basic flush that exclusively flushes the L1 data and instruction caches, the TLBs, and the branch predictors. Through their evaluation they find that further stateful components - such as the LFSR generating a pseudo-random index sequence for the cache replacement policy and the round-robin memory arbiters of the core - still leak data through the microarchitectural state. As such, they also implement a Full Flush, which clears these components along with those covered by the basic flush. The `fence.t` instruction proceeds as follows:

- Save the program counter.
- Save locally modified (dirty) architectural state.
- Drain pending transactions.
- Clear components that are not cleared on reset.
- Assert Microreset. Clearing all flip-flops containing non-architectural state.

A full explanation of each step is provided in [17].

Given this implementation, they augmented the seL4 kernel to call the `fence.t` instruction on every context switch, finding an overhead of around 0.7%. They note that this should be seen as a pessimistic result due to the use of idle tasks, and that in a setting with high memory contention the latency may be almost entirely hidden.

2.5.1 With software support

While `fence.t` was effective and inexpensive on simple in-order cores, a follow-up paper [18] argues that out-of-order execution CPUs introduce challenges for the implementation. They address these challenges by presenting a software-supported temporal fence `fence.t.s` methodology. They show that this software-supported version is able to close all on-core timing channels without measurable hardware overhead, and with a minimal average performance impact of 1.0%.

2.6 FreeRTOS

FreeRTOS is a class of real-time operating systems (RTOS) designed to be small enough for microcontrollers, while not limiting its application to microcontrollers alone. Out of the box, FreeRTOS provides core real-time scheduling functionality, inter-task communication, timing and synchronisation primitives, along with other optional add-on features [20]. It does this through the use of a priority-based scheduler, which is inherently not fair and can starve lower priority tasks.

FreeRTOS-MPU is a memory-protected version of FreeRTOS, which at the time of writing is supported on select ARM-based microcontrollers. It enables tasks to run in either privileged or unprivileged mode and restricts access to resources such as RAM, executable code, peripherals, and memory beyond the limit of a task's stack [21].

A third-party port of FreeRTOS-MPU to the RISC-V ISA currently exists [22]. This port forms the basis for this thesis's implementation. There are several reasons for this choice. The first and most important is familiarity: I have previously worked with the RISC-V ISA, which reduces the overhead of adopting a different ISA. Second, the related work on timing channels generally focuses on RISC-V, so future comparisons between implementations are more relevant if all are built on the same platform. Finally, the `fence.t` instruction introduced in Section 2.5 is designed with the RISC-V instruction set in mind. While the underlying idea could be implemented on other platforms, work on hardware support for those platforms is not as far along. Therefore, assuming this implementation will eventually run on real hardware, RISC-V seems the most reasonable choice.

3 Related Work

Other works has been done in attempting to close potential timing channels. These fall into two primary groups. User level mitigations and kernel level mitigations. This section will first look into the FaCT DSL language, which is a user level mitigation strategy against timing channels, and then it will present two microkernels each with a slightly different approach to closing timing channels.

3.1 User-level mitigations

The argument for using user level mitigations stems from the definition of time protection. Time protection is defined within [9] as "A collection of OS mechanisms which jointly prevent interference between security domains that would make execution

speed in one domain dependent on the activities of another". An OS, which implements Time protection must ensure that all execution within partitioned parts does not depend on each other making both covert channels and side-channels infeasible. This is quite a strict version of time protection, and to close covert channels it must be, but quite a significant number of timing channels can be closed at the user-level if one allows a distinction between different information types.

One such example is the FaCT DSL [6]. FaCT sought to address the challenges of writing constant-time cryptographic code, which does not depend on "secret" information. That is, instead of treating all information as possible leakage, one indicates what information must not be leaked through a timing channel, and the FaCT type system guarantees that in isolated execution this will not leak. FaCT works by making the code constant time in regard to the sensitive information, and any branching is only dependent on non-sensitive information. The idea being that any attack can not measure the execution time of the encryption code to gather information about sensitive information. This does work to an extent, by guaranteeing constant time at the C program level, but compiler-introduced and variable execution times of instructions based on inputs can still leak sensitive information. For example, the division operator is notoriously known to have its execution dependent on the numerator [23], [24]. Mitigations against these attacks would still require help from hardware, and make use of specific instructions that guarantee a constant execution time.

Furthermore, information that the operating system holds about a given process could also be deemed sensitive. For example, in highly secure environments the runnability of a task could be deemed sensitive, and no changes on the user level side will ever close such timing channels, as the OS scheduler will unknowingly always leak this information through its scheduling decisions. Therefore, while user level implementations do help significantly, the issue can not be fully closed without both the support of the OS and hardware.

So while user level mitigations are effective at closing a significant amount of timing channels, it is not capable of providing complete time protection by itself, and still requires help from the OS and hardware to do so. The user level mitigations are thus not useable in the context of full time protection, but could be used in cases where restrictions are more lenient.

3.2 The missing OS Abstraction: seL4

The paper Time Protection: The missing OS Abstraction [9] presents a customized version of the seL4 microkernel that aims to protect against all possible timing

channels in the OS.

3.2.1 Threat model

The paper's threat model consists of a victim program containing a Trojan that uses any means necessary to leak information through timing channels. The only restriction on the Trojan is that it must run in a separate domain from the receiver/spy program.

Specifically, they differentiate between two types of channels: the **covert** channel and the **side** channel. They argue, that by protecting against the covert channel, it immediately follows that the side-channel is also protected.

3.2.2 Channels

Two main categories of channel are referenced throughout the paper. On-core state - including L1 and L2 caches, Translation Lookaside Buffers, and other microarchitectural structures - which is unique to each core, and shared state, which includes higher-level caches that are shared between different cores. To protect against all channels, they provide the following five requirements:

- *Flush on-core state:* When time-sharing a core, all on-core microarchitectural state must be flushed on every domain switch, unless the hardware supports specific partitioning of this state.
- *Partition the OS:* Each security domain must have a private copy of OS text, stack and global data. That is, all the dynamic OS kernel memory must have the same separation as userland execution.
- *Deterministic data sharing:* Access to any remaining shared OS data must be sufficiently deterministic to avoid information leakage.
- *Flush deterministically:* State flushing must be padded to its worst case latency.
- *Partition interrupts:* When sharing a core, the OS must disable or partition any interrupts other than the preemption timer. (This is only a concern intra-core).

3.2.3 Cloning the OS

The paper introduces a method for cloning the OS. A userland program with the capability to clone gives up a subset of its own allocated memory for the clone call, and the kernel then creates a kernel image copy in the user-supplied memory. This

memory is then marked as kernel data, meaning that userland cannot modify it, but any system calls will run through this kernel copy.

This mechanism leverages seL4's security domains. The initial kernel creates a master Kernel_Image that represents the current kernel and includes the clone right. It hands this capability, along with the image size, to the initial user process, which then partitions the memory into security domains and clones the OS into each of these new domains.

The cloning process consists of three steps:

1. The user process retypes some Untyped memory into an uninitialized Kernel_-Image and Kernel_Memory of sufficient size.

2. It allocates an address space identifier to the uninitialized Kernel_Image.

3. It invokes Kernel_Clone on the new Kernel_Image.

3.2.4 Cache coloring

To protect caches that are too large to be flushed, cache coloring is implemented. This ensures that separate security domains cannot access the same parts of cache, eliminating mutual influence. Cache coloring reduces the total amount of available cache space for each domain, meaning that the process creating the security domains must account for this factor so that the distribution of cache space is sufficient for all domains. A more in depth explanation about cache coloring and other hardware specific items are referenced in Section 7.5

3.3 S3K

S3K is a real-time operating system that provides a capability-based multicore partitioning kernel for embedded RISC-V systems. It provides spatial and temporal isolation, time protection, and dynamic resource reconfiguration. Furthermore, S3K's scheduler guarantees deterministic process dispatch, free from microarchitectural interference [8].

It achieves this through three main attributes:

- A capability model.

- A time-driven, capability-based scheduler with systematic use of `fence.t`, worst-time padding, and constant-time kernel operations.

- Predictable, constant-time system calls.

3.3.1 Implementation

The implementation provides a minimal trusted computing base (TCB) with strong isolation and security guarantees. The basic execution unit in S3K is a *process*, a self-contained entity whose resources are defined by its capabilities. These processes are designed to host bare-metal real-time operating systems or applications, with the kernel acting as a bare-metal hypervisor. The maximum number of processes is statically defined at compile time, which is the foundation of the time slices in the kernel. Processes use *monitor capabilities* and capability revocation to manage other processes, reclaim resources, and repurpose existing processes for new tasks [8].

Capabilities: Every S3K capability operation has a bounded worst-case execution time (WCET) and is free of timing side-channels. Capabilities are implemented in software and stored in a capability derivation tree (CDT), which resides in a statically allocated capability table (ctable) in memory.

Scheduling: S3K's scheduler is a partitioned scheduler that divides time into *major frames*, which are further partitioned into *minor frames*. S3K treats minor frames as first-class capabilities, enabling dynamic reconfiguration of computing resources while maintaining process isolation [8].

A major frame is divided into a fixed number of equal duration time slots. A single time slot represents the smallest scheduling unit. A time slice capability grants control over a range of time slots on a hardware thread (hart). The process that owns the time slice capability receives a minor frame of its own, whose length is less than or equal to the major frame. Each time slice capability has an *enable* field that specifies whether the minor frame is active. If set, the owner of the time slice is scheduled during the corresponding minor frame; otherwise the minor frame represents idle time.

The kernel is in principle non-preemptable. It uses preemption points to ensure that the WCET is bounded. The kernel checks for preemption at the start of every system call and aborts the system call if preemption is required.

By the definition of domains in S3K, processes from different domains are never scheduled in parallel. Thus, during a domain switch, all running processes of the current domain are preempted, invoke the scheduler, and are subsequently delayed by the temporal fence.

3.3.2 Time protection

S3K uses the temporal fence instruction at context switches to flush the core-local microarchitectural state.

S3K is designed to reside entirely within the small scratchpad memory (SPM) backed by the core-local L1 cache. This enables the kernel's microarchitectural footprint to be efficiently flushed and avoids reliance on larger caches.

S3K only protects a process's in-kernel state from side-channels; it does not protect a process's user-level state. Each process is responsible for protecting its own user-level execution from side-channels in higher level caches. This is different from the approach in [9], where cache coloring guarantees that processes in different domains never access the same parts of the cache, thereby preventing side-channels even in larger caches.

All system calls are designed to be non-interfering and constant-time with respect to the invoking process's authorized observations; that is, all control flow decisions and memory accesses depend only on data the process is permitted to observe.

3.3.3 Evaluation

They evaluate the implementation across several factors, the most significant for this thesis being scheduling jitter and side-channels. They first find that Inter-Process Communication (IPC) operations have the highest recorded non-preemptable cost of 1,195 CPU cycles. Together with an estimated 16,000 cycles for data cache write-back during the temporal fence, they set the CSPAD to 18,000 cycles for the `fence.t` instruction. They make use of these findings for the WCET padding throughout the codebase.

Scheduling Jitter: A measurer process, scheduled once per major frame, records the *mcycle* performance counter value upon dispatch. To simulate interference, they introduce interfering processes that execute random system calls and pollute the cache. They find that when the measurer exclusively occupies the SPM, the other processes have no impact on the measurements. However, when the measurer occupies DRAM, noticeable jitter appears. They note that in some cases this jitter originates from the small portion of code in the measurer that records the dispatch time, rather than from the scheduler itself. When more than one process is scheduled at a time, jitter also appears in the scheduler.

Side channels: They implement a Trojan process and a spy process. The Trojan signals a "0" by remaining idle and a "1" by behaving like the interfering processes

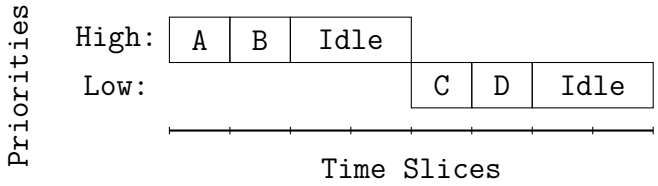


Figure 1: Time protection within same priority

from before. The spy then measures jitter, system call execution times, and cache access times to infer the Trojan's signal.

They find that when the measurer and Trojan both reside in DRAM, there is a near perfect information flow. In contrast, when both reside in the SPM, there is no measurable amount of information flow.

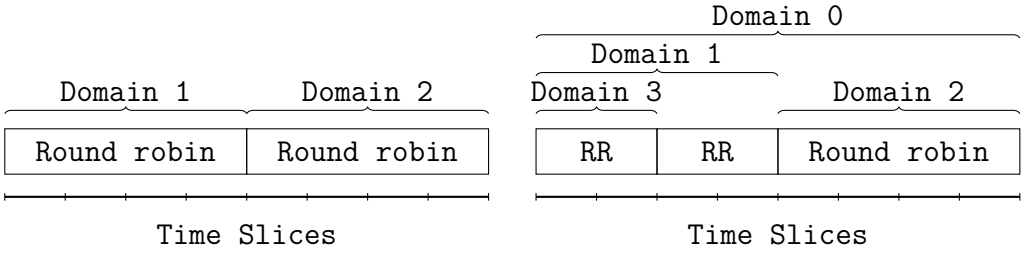
4 Scheduling analysis and Design

One inherent timing channel within FreeRTOS is the scheduling policy itself. This section examines the default fixed-priority preemptive scheduler within FreeRTOS and explores various theoretical approaches for closing scheduler based timing channels. By evaluating the trade-offs between the different approaches, this analysis provides the foundation for the implementation of the domain-level scheduler implemented in this thesis.

4.1 Closing Scheduling Based Timing Channels

There are a few approaches to closing the different timing channels, each with their own pros and cons. The following presents the most notable approaches theorised during this thesis.

Removing time: One approach to solve a lot of the timing channels would be to disallow tasks to read the current time. This in theory would remove the scheduling based timing channel, but this solution would most likely entail quite a significant decrease in the possibilities of tasks, as any action, which could leak time information to the tasks would need to be prohibited. As such, this approach was not considered further.



(a) Static domain scheduler (b) Dynamic domain scheduler

Figure 2: Domain scheduling

Protection within priority: If the goal is to keep a sense of priority based scheduling, then it should be possible to guarantee timing protection for tasks of same priority or tasks of different priorities. Say for example, each priority has a known array which constitutes the time slices for the given priority. When the scheduler attempts to schedule a task, then it prioritizes the higher priority array if a task is ready within, and uses time slice based scheduling within the priority. An example of this can be seen visualized in Figure 1, where tasks A and B are within the same priority. Here, if A and B are assumed runnable, then they would be scheduled before the lower priority C and D. This would allow C and D to gather information about the runnability of the entire High priority noting that either A or B was runnable if C or D are delayed. However, within the same domain tasks A and B cannot gather information about each other. Say for example A gets to run, then it does not know whether task B is runnable or not, as the domain is always running the entire time slice defined by the aforementioned array.

This approach would by no means close all timing channels, as mentioned cross priorities could still leak timing information. Other constraints with this approach are that they limit the number of possible tasks for given priorities. In the previous example 4 time slices were provided within the High and Low priorities, and one would not be able to fill more than these 4 time slices as this would then change the total time slice when running a specific priority and thus a task within the given priority could gather information about the total number of time slices. Another issue is that one cannot simply give up CPU time. When a given task wishes to sleep for a short duration, the time slice must be replaced with idle time, which significantly reduces overall CPU utilization. Once a task is completely finished it is possible to then take back the time slice, but the new task that would be scheduled in this slot would then gather the information that the previous task finished its execution.

Domain scheduling: Another approach is the one seen in [9], which is the domain-level scheduler. The general idea is quite similar to the protection-within-priority approach discussed above, but the idea of priorities is removed entirely and instead the focus is on domains. Each domain can have multiple tasks within, where timing channels are not a concern, and time protection is only enforced between different domains. In Figure 2a an illustration of this scheduler can be seen. The idea is that at compile time, the different domains and their respective time slice allocations are known (in the example that would be 4 for both domain 1 and 2). The domain scheduler then takes care of scheduling the domain, and guarantees time protection only between the different domains. This approach is still inhibited by the need to know the total number of time slices at compile time, and further it also needs the number of time slices for each domain at compile time. However, where this benefits over the previous approach is, that within each domain there is a lot more freedom to do as one pleases. For example, one could imagine having a regular round-robin scheduler within a domain, which then allows more CPU utilization within this specific domain and less unused resources.

An extension of this approach would be the one seen in Figure 2b, which closely follows the capability based scheduler in [8]. The idea is that within a specific domain, one can give up some of the attributed time to create a new subdomain. This would mean that there is time protection between subdomain 1 and the rest of domain 1[1]. In theory it should be possible to give up these time slices back to the domain-level scheduler, which would then treat this as a new domain.

This approach seems by far the most flexible and most robust in terms of real world applications. As such, this thesis attempts to implement this domain scheduler within the FreeRTOS kernel.

It is important to note that this approach does not remove all timing channels. Each domain uses a priority based scheduler, which as discussed in Section 2.3 is susceptible to timing channels. The idea is instead that such channels are allowed in tasks where protection is not relevant and throughput is valued higher. Instead, when one explicitly wants a safe execution environment a domain can be created for this, which is then protected after initialization.

[1]Other than creation and finishing assuming a domain is allowed to give back the time slices upon finishing, which would leak timing information

5 Implementation

Moving from theoretical analysis to a functional secure system requires changes within the existing FreeRTOS framework. This section provides an in-depth explanation of the specific architectural modifications required to provide the domain-level scheduler discussed in the previous section. It aims to cover the extensions of the FreeRTOS-MPU API, the introduction of new configuration flags, and the core logic behind the domain-level scheduler. The specifics about how constant-time domain creation was implemented, while keeping the functionality of the priority based scheduler within each domain, is also presented.

To guide the reader through these structural changes, it is necessary to first define a few foundational terms used throughout this section. A *domain* refers to an isolated execution environment containing its own security boundaries and a dedicated priority-based scheduler. The global execution timeline is partitioned into discrete units of time called *time slices*, which are dynamically assigned to domains. Within these domains, individual tasks are managed via extended versions of the standard FreeRTOS *Task Control Block* (TCB).

The remainder of this section details the concrete implementation modifications required to the domain-level scheduler. First, Section 5.1 explains how the FreeRTOS-MPU API is extended to pass domain-specific parameters during task creation. Next, Section 5.2 introduces the new compile-time configuration flags necessary to control compile time options for the scheduler. The underlying mechanics of domain switching, machine timer interrupts, and microarchitectural flushes are then explored in Section 5.3. This is followed by an explanation of the array-based index arithmetic that enables constant-time domain creation in Section 5.4. Finally, Section 5.5 details the multi-dimensional array transformations required for the functionality of the intra-domain priority scheduler, before Section 5.6 discusses the adjustments made to allow for an intended communication channel between domains.

5.1 Extending the api

To implement the domain-level scheduler I must first incorporate the idea into the existing FreeRTOS framework. With FreeRTOS MPU, I am able to create unprivileged tasks which have restricted access to syscalls and provide them with a memory space that is partitioned. For this, the general FreeRTOS api is extended with a multitude of new functions most importantly for this project is the `xTaskCreateRestricted`[21]. The idea behind the domain scheduling fits with the purpose of memory protection, and as such it seems the obvious choice for modification to extend it with the domain

functionality. The signature of the aforementioned function is provided in the listing below:

```
typedef struct xMEMORY_REGION
{
    void * pvBaseAddress;
    uint32_t ulLengthInBytes;
    uint32_t ulParameters;
} MemoryRegion_t;

typedef struct xTASK_PARAMETERS
{
    TaskFunction_t pvTaskCode;
    const char * pcName;
    configSTACK_DEPTH_TYPE usStackDepth;
    void * pvParameters;
    UBaseType_t uxPriority;
    StackType_t * puxStackBuffer;
    MemoryRegion_t xRegions[ portNUM_CONFIGURABLE_REGIONS ];
    #if (
      ( portUSING_MPU_WRAPPERS == 1 ) &&
      ( configSUPPORT_STATIC_ALLOCATION == 1 )
    )
        StaticTask_t * const pxTaskBuffer;
    #endif
} TaskParameters_t;

BaseType_t xTaskCreateRestricted(
  const TaskParameters_t * const pxTaskDefinition,
  TaskHandle_t * pxCreatedTask
) PRIVILEGED_FUNCTION;
```

Listing 1: xTaskCreateRestricted function signature

That is, one can create a task which has access to a number of memory regions defined by the region start address (pvBaseAddress), the region size in bytes (ulLengthInBytes) and the access permissions for the region (ulParameters). The function parameter uxPriority is also of importance. This function is only callable by privileged tasks, which restricts the freedom when creating new domains, something which will be referenced in future sections.

The idea is to extend this function signature as follows:

```
...

```

```
typedef struct xDOMAIN_PARAMETERS
{
  uint32_t ulSliceIndex;
  uint32_t ulSliceLength;
} DomainParameters_t;

typedef struct xTASK_PARAMETERS
{
  ...
    #if ( configENABLE_DOMAINS == 1 )
        DomainParameters_t * xDomainParameters;
    #endif
} TaskParameters_t;

BaseType_t xTaskCreateRestricted(
  const TaskParameters_t * const pxTaskDefinition,
  TaskHandle_t * pxCreatedTask
) PRIVILEGED_FUNCTION;
```

Listing 2: Extended parameters and signature

A task with the capability to create a new restricted task would then have to provide the index in the global time slice array, along with the length of the new domain that will be created. If no xDomainParameters is passed to the function (i.e. the pointer is NULL) this will be treated as creating a task within the same domain, and will be handled by the per domain priority based scheduler.

With this, no more of the external api has to be changed.

5.2 New configuration flags

To enable the functionality of domain-level scheduling, a few new compile flags need to be introduced.

configENABLE_DOMAINS A binary flag to indicate whether or not to use the domain-level scheduler alongside the default priority based scheduler.

configNUM_TIME_SLICES The total number of time slices for the domain scheduler to take into account. As discussed in Section 4, this flag also constitutes the maximum number of possible domains, such that any domain scheduling decisions can be done in constant time.

configNUM_TICKS_PER_SLICE The number of timer interrupt ticks to constitute a single time slice. Say that the ticks is defined as `#define configTICK_RATE_HZ ((TickType_t) 1000)` with `#define configNUM_TICKS_PER_SLICE (5)`, this would mean that a single time slice is equal to TIME_SLICE $= \frac{1s}{1000} \cdot 5 = 5ms$.

5.3 Domain switching

To keep to the fundamentals of FreeRTOS the idea is to keep the functionality of the priority based scheduler within FreeRTOS but internally in a domain. That is, within a specific domain the same priority based scheduler is available. This in turn means that if no new domains are created, the functionality of the scheduler should be nearly unchanged[2].

FreeRTOS makes use of a single static file `tasks.c` for the development of the priority based scheduler. In this thesis I am only focused on the single core implementation and while work on the multicore version would be interesting, that is left for future work.

In the MPU version of FreeRTOS when a machine timer interrupt occurs it is handled in the following manner:

```
freertos_risc_v_mtimer_interrupt_handler:
    portcontextSAVE_INTERRUPT_CONTEXT
    portUPDATE_MTIMER_COMPARE_REGISTER
    call xTaskIncrementTick
    beqz a0, exit_without_context_switch
    call vTaskSwitchContext
```

Listing 3: FreeRTOS mtimer interrupt handler

It increments a global tick, and switches the task context. However, FreeRTOS only increments the global `xTickCount` if the priority based scheduler is running and not suspended for critical handling. This means, I have to introduce my own proxy for Ticks, as the per domain `xTickCount` would leak information about other scheduler states and possibly delay the domain switching. For this purposes I can hook into the pre-existing `xTaskIncrementTick` function, and check for a domain switch.

```
/* Static variable to contain the current Domain Tick */

PRIVILEGED_DATA static size_t pxCurrentDomainIndex = 0;
```

[2]There will be some overhead with the implementation of course.

```
PRIVILEGED_DATA static volatile TickType_t xEffectiveTick = ( TickType_t ) configINITIAL_TICK_COUNT;
PRIVILEGED_DATA static volatile TickType_t xDomainTick = ( TickType_t ) 0;

BaseType_t xTaskIncrementTick( void ) {
...
const TickType_t xConstTickCount = xEffectiveTick + ( TickType_t ) 1;

/* Increment the effective tick. This is different than xTickCount,
 * which does not increment if the scheduler is in a critical section.
 * The Domain scheduler does not care about critical section */
xEffectiveTick  = xConstTickCount;

/* Update the Domain tick in regards to the effective tick */
xDomainTick = (xEffectiveTick / configNUM_TICKS_PER_SLICE) % configNUM_TIME_SLICES;

/* Time slot expired or wrap-around */
if( xDomainTick >= pxCurrentDomainIndex + xDomains[pxCurrentDomainIndex].uxLength
    || xDomainTick < pxCurrentDomainIndex )
{
    /* Save current task to logical domain info */
    xDomainInfo[xDomains[pxCurrentDomainIndex].uxDomainID].pxPreviousTCB = pxCurrentTCB;

    xSwitchRequired = pdTRUE;

    /* Flush on domain switch */
    temporal_fence_t();

    /* Contiguous partition: next block starts exactly here */
    pxCurrentDomainIndex += xDomains[pxCurrentDomainIndex].uxLength;
    if( pxCurrentDomainIndex >= configNUM_TIME_SLICES )
    {
        pxCurrentDomainIndex = 0;
    }

    /* Restore previous task from logical domain info */
    TCB_t * previous_tcb = \
      xDomainInfo[xDomains[pxCurrentDomainIndex].uxDomainID].pxPreviousTCB;
    /* First time, use idle task */
    if (previous_tcb == NULL) {
        pxCurrentTCB = xIdleTaskHandles[ 0 ];
    } else {
        pxCurrentTCB = previous_tcb;
```

```
    }
}
...
}
```

Listing 4: Custom hook for xTaskIncrementTick

As such, whenever there is a machine timer tick, the code calculates in constant time the time slice (xDomainTick) and checks whether a domain switch should occur. If the check finds that a new domain should be scheduled, then the data is switched to the new domain and a temporal flush is performed to clear the microarchitectural state. The flush happens in such a manner that any data accessed in regard to the new domain is non-interfering with the previous domain. Once a switch has been decided, only data that is within the incoming domain is retrieved following the flush, this means that the new domains execution will not be based on the microarchitectural state of the previous domain.

5.4 Constant time domain creation

To keep track of the different domains new global constants are defined:

```
typedef struct DomainBlock {
  UBaseType_t uxDomainID; /* This time slices domain ID */
  size_t uxLength; /* Length of this domain */
  size_t uxNext;
} DomainBlock_t;

/* Timeline over configNUM_TIME_SLICES, indexed by the start of a block */
PRIVILEGED_DATA static DomainBlock_t xDomains[configNUM_TIME_SLICES] = {
    [0] = {.uxDomainID = 0, .uxLength = configNUM_TIME_SLICES, .uxNext =
    ↪ SIZE_MAX}
};
```

Listing 5: Domain timeline

For now the main data structure introduced is an array, namely xDomains, storing the timeline. This array is indexable by the start time of the current group of time slices and contains information about which domain owns the group and the length. Two example states of this array can be seen visualized in Figure 3. For the case where Domain 0 owns all time slices xDomains would only contain a single valid index, namely the 0 index, which stores that it is domain 0, that owns the group of time slices and the group has a length of 6.

(a) Domain 0 owns all time slices

(b) Domain 0 is fragmented when creating another domain

Figure 3: Example timelines when creating domains

This way, the domain switching can be directly indexed once a new domain switch is needed, providing constant time retrieval of the next domain.

At first glance domain creation also seems simple. In Figure 3a there are three main cases for domain creation. Either domains are created by giving up time slices at the start or the end of the current time slice, which would simply create two separate domains with no ”split” domain. If instead a domain is created in the middle of the current domain, then the current domain is split and might be scheduled several times during a single round trip of the time slices. Such an example can be seen in Figure 3b, where Domain 0 runs at separate times of execution. This poses an issue when wanting to create yet another domain. As mentioned in Section 5.1, a user of the OS would provide the time slice for which to start a new domain and the length of the domain. Given that not all indices throughout the domain are defined, the array would need to be scanned from left to right to figure out if the current domain truly does ”own” the given time slices the user is attempting to access. If a user wants to create a new domain from Domain 0s time slice 4-6 (0 indexed), then the naive method would loop from the start of the array until reaching the group of time slices, which contains these time slices, namely indices 4-8. This execution is not constant time with respect to what domain 0 should know as the execution time of the loop depends on how many other domains exist.

To solve this issue the `uxNext` field was introduced, which contains the index of the next group of time slices owned by the current domain. This field is used as a singly linked list, which can be looped over to find the encompassing group of time slices. Given this only loops over this domains owned group of time slices, it does not leak information about the total number of domains as before. In the case that there is no next domain the `SIZE_MAX` is used.

With this in place, it allows finding the encompassing domain as follows:

```
1 static BaseType_t prvFindContainingBlock(
2     size_t uxFirstBlock,
```

```
    size_t start,
    size_t length,
    size_t * puxPrevBlock,
    size_t * puxBlock)
{
    size_t prev = SIZE_MAX;

    for (size_t current = uxFirstBlock; current != SIZE_MAX; current =
    ↪ xDomains[current].uxNext)
    {
        size_t blockEnd = current + xDomains[current].uxLength;

        if (start >= current && (start + length) <= blockEnd)
        {
            *puxPrevBlock = prev;
            *puxBlock = current;
            return pdTRUE;
        }

        prev = current;
    }

    return pdFALSE;
}
```

Once the encompassing group of time slices is found, the new domain must be created, which is done via index arithmetic as seen below:

```
...
/* Right fragment */
size_t rightStart = SIZE_MAX;
if (start + length < blockEnd)
{
    rightStart = start + length;
    xDomains[rightStart].uxDomainID = uxCurrentDomainID;
    xDomains[rightStart].uxLength = blockEnd - rightStart;
    xDomains[rightStart].uxNext = xDomains[blockStart].uxNext;
}

/* Left fragment: shrink existing block */
if (start > blockStart)
{
    xDomains[blockStart].uxLength = start - blockStart;
    if (rightStart != SIZE_MAX)
    {
```

```
        xDomains[blockStart].uxNext = rightStart;
    }
}
else /* No left fragment: remove blockStart from parent's list */
{
    size_t spliceNext = (rightStart != SIZE_MAX) ? rightStart :
    ↪ xDomains[blockStart].uxNext;
    if (prev == SIZE_MAX)
    {
        xDomainInfo[uxCurrentDomainID].uxFirstBlock = spliceNext;
    }
    else
    {
        xDomains[prev].uxNext = spliceNext;
    }
}
...
```

First, the code checks whether a new right fragment of the current domain would be created. It would exist if the code is attempting to create a new domain whose start ends before the current domain ends. If such a fragment exists, then the index in `xDomains` is updated to track it.

Next, a check determines if a left fragment is created, and simply shrinks the encompassing time slice group, updating the `uxNext` field to indicate that the domain has been split and the next block owned by the current domain is that of the new right fragment.

If no left fragment is created, then a new domain is being created at the start of the encompassing block, so the encompassing block start is removed from the linked list, and updated to point to the right fragment if one was created or the next block. In the case that there is no next block, then it was previously set to SIZE_MAX, indicating again that there is no next block.

Finally, the `xDomains` array must be updated to include the newly created domain, which is simply done as follows:

```
...
UBaseType_t uxChildDomainID = uxNextDomainID++;
xDomains[start].uxDomainID = uxChildDomainID;
xDomains[start].uxLength = length;
xDomains[start].uxNext = SIZE_MAX;

xDomainInfo[uxChildDomainID].pxPreviousTCB = NULL;
xDomainInfo[uxChildDomainID].uxFirstBlock = start;
```

```
...
```

The global xDomainInfo is the array used for scheduling the TCB within each domain, and is indexable by the domain id rather than the start of the time slice block.

5.5 Per domain scheduling

For the domain-level scheduler to work with intra-domain priority based schedulers, another dimension must be added to all of the items. That is, all variables that previously were a constant would have to be an array with length equal to the number of time slices and same extension for previous arrays. All accesses to these constants/arrays would have to follow a similar transformation, where the currently scheduled domain would choose the correct values for the priority based scheduler to function correctly. A few examples of these transformations are shown below:

```
/* Non-domain scheduler implementation*/
PRIVILEGED_DATA static volatile BaseType_t xSchedulerRunning = pdFALSE;
PRIVILEGED_DATA static volatile TickType_t xPendedTicks = ( TickType_t ) 0U;

/* Domain scheduler equivalent */
PRIVILEGED_DATA static volatile BaseType_t
    ↪ xSchedulerRunning[configNUM_TIME_SLICES] = { pdFALSE };
PRIVILEGED_DATA static volatile TickType_t
    ↪ xPendedTicks[configNUM_TIME_SLICES] = {( TickType_t ) 0U};
```

Listing 6: Transforming the priority based scheduler

Note that while the first element is initialized here, the initialization of the full items are moved to a separate function, as all of the schedulers would have to be initialized in the same fashion as the default scheduler.

With this, the general foundation of the domain-level scheduler is functional, and the standard `mpu_blinky.c` example works given all tasks run within the same domain.

5.6 Cross domain communication

As it stands, the FreeRTOS api has next to no information about the different domains, meaning that items such as the Queue used for message transferring between domains is not available. Generally this is a good thing, as by default there is domain isolation,

as any internal handling within tasks.c makes use of the current domain. This restriction removes intended communication paths between domains. In FreeRTOS communication happens through Queues, where messages can be passed. The general workflow for Queue communication in the MPU version of FreeRTOS is as follows:

- Create Queue kernel object in a priority task.
- Grant access to the Queue object to any user level tasks, that should communicate through the Queue.
- One task sends messages through `xQueueSend`.
- Another task receives messages through `xQueueReceive`.

For the cross domain communication the workflow should be the same, but it is the internal handling that has to change. Currently the internal handling of the `xQueueSend` is:

- Check there is room in the queue for the message.
- Check if there is a task waiting to receive on the Queue.
  - If yes, move the task to the ready list.
  - If no, do nothing.

This is generally also fine, except that when moving a task to the ready list, the task might be outside the domain that is currently running. When the Queue moves a task to the ready list, it does so through the function `xTaskRemoveFromEventList`. This function takes a linked list of tasks, namely the tasks waiting on the queue, removes the first task from the list and moves the given task into the ready list if the scheduler is suspended and otherwise moves it to the `xPendingReadyList` to wait for the next scheduler suspension before moving into the true ready list.

This is where changes must be made to allow for cross-domain communication. First, each task needs some way of tracking the domain it resides within, which is easily done by extending the `TCB_t` struct with an extra field:

```
1 typedef struct tskTaskControlBlock
2 {
3     ...
4     #if ( configENABLE_DOMAINS == 1 )
5         UBaseType_t uxDomainID;
6     #endif
```

```
} tskTCB;

typedef tskTCB TCB_t;
```

The Queue can now reference this DomainID when modifying the priority-schedulers and as such adds the tasks to the correct arrays.

Given this communication, the `mpu_blinky.c` example can now be created with different domains communicating. It is important to note here that there is, of course, some timing leakage introduced between these two domains, as they can time the response time of different messages. Here it is up to the individual domains to respond in a deterministic manner so as not to leak any secret information.

6 Evaluation

To verify the effectiveness and performance of the domain-level scheduler, two primary scenarios were simulated. The first example is a blinky demo, where two tasks are created in separate domains. The two tasks are given access to a Queue object, which they can use for communication. The demo is meant to demonstrate proper means of cross domain communication. The second scenario is a larger demo, which create 4 different domains, each with a single privileged task. This task then creates a new user level task with a higher priority than itself. The idea here is to show how the functionality of the within domain priority based scheduler still functions as expected, while the domain-level scheduler keeps a deterministic scheduling between the domains. These scenarios are evaluated using the QEMU system emulation with instruction based counting to provide for deterministic execution for the guest system, which in this case is the modified FreeRTOS kernel.

These scenarios are evaluated using QEMU (Quick Emulator), an open-source machine emulator and virtualizer. QEMU allows for full system emulation of the target RISC-V architecture, enabling the modified FreeRTOS kernel to run without physical hardware. However, QEMU does not natively provide cycle-accurate emulation, instead it defaults to the host system clock virtual interrupts, meaning that execution time can be influence by the host system.

To achieve a reliable baseline, the evaluation utilizes QEMU's `-icount` flag, which provides deterministic execution from the guest point of view. Instead of making use of the host interrupt, QEMU instead makes use of instruction counting to determine elapsed time [25]. While this by no means is cycle-accurate and does not take into account any microarchitectural state (such as cache misses), it can be used to approximate the time each domain is allowed to run before a scheduling occurs.

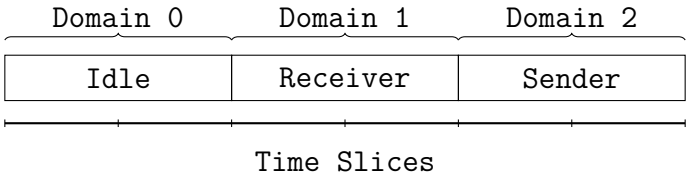


Figure 4: Timeline of the Blinky Domain Scheduling

6.1 The blinky example

The RISC-V mpu port includes a standard `mpu_blinky.c` example. The example program creates two tasks, a sender and a receiver. The sender adds a message to a `Queue` at a set interval, and the receiver reads from the `Queue` whenever there is a message available. After a set amount of message passing, the sender reads from a restricted memory address causing a pmp fault that gets handled by a custom exception handler. For cross-domain communication, a similar approach can be used to check that the tasks are still capable of passing messages to each other. The domain-level scheduler is set up such that the main function is called with elevated privilege and it is capable of creating queues and new tasks. The tasks creation is modified to make use of the new api, such that it creates two new tasks, each in a separate domain. The scheduling timeline of this modified example is provided in Figure 4.

That is, when the initial program is started and execution is within domain 0, then domain 0 has given up 2 time slices for the Sender (domain 2), and 2 time slices for the Receiver domain (Domain 1). As it itself finishes execution, the rest of its time slices will simply be used by the idle task. Each task is capable of reading the current domain tick, and the demo simply prints this, when execution starts. Running the demo produces the following snippet of output:

```
Sending to task, DomainTick: 4
Message received from task, DomainTick: 2
Sending to task, DomainTick: 4
Message received from task, DomainTick: 2
Sending to task, DomainTick: 4
Message received from task, DomainTick: 2
Sending to task, DomainTick: 4
Message received from task, DomainTick: 2
Sending to task, DomainTick: 4
prvQueueSendTask: trying to access privileged data

```

```
=== PMP ACCESS FAULT ===
mcause  = 0x00000005
mepc    = 0x80003730
mtval   = 0x80180001
mstatus = 0x00000080
type    = load
task    = Tx
system halted.
```

The output does show some interesting properties with regards to the domain scheduler. First, although the receive task is scheduled before the sender task, given that there is no message to read in the Queue, it simply sleeps for its very first execution, scheduling the idle task. Next, one can see that the sending happens always at the specified DomainTick 4, and that the receiving also happens within the expected domain tick 2. Another interesting note is, that in the very last iteration before the access fault, the receiver task is never able to receive the final message, as the sender continues uninterrupted before causing the fault.

6.1.1 Constant time

While the previous example shows that the Domain Tick seems constant for both[3], it does not show that each domain runs for a constant amount of time before interruptions. To truly show that this runs in constant time, the evaluation is limited by the tools available. As mentioned, QEMU does not provide cycle-accurate emulation and so instead the following is evaluated using instruction based counting.

In RISC-V the `rdcycle` instruction reads XLEN bits of the `cycle` counter, which holds a count of the number of clock cycles executed by the processor core on which the hart is running from an arbitrary start time in the past. Given FreeRTOS runs with XLEN 32-bit, it is necessary to also make use of the `rdcycleh` instruction, to read the upper 32 bits of the cycle counter, to get the true cycle count. This means that one can find a round trip time of the domain switch in the following fashion:

```
static uint64_t ullPrevMtime = 0;

uint64_t ullMtime;
uint64_t ullMtimeHigh;

asm volatile ("rdtime %0" : "=r"(ullMtime));
asm volatile ("rdtimeh %0" : "=r"(ullMtimeHigh));
ullMtime = ((ullMtimeHigh << 32) | ullMtime);
```

[3]If it varies then the likely culprit is the priority based scheduler within each domain.

By saving the previous cycle count from when this point was last reached and calculating the round trip time, the demo can be made to print this round trip time for each domain, producing output like the following:

```
...
Sending to task, DomainTick: 4
Round Trip: domain=2 delta=50000
Round Trip: domain=0 delta=50000
Message received from task, DomainTick: 2
Round Trip: domain=1 delta=50000
Round Trip: domain=2 delta=50000
Round Trip: domain=0 delta=50001
Round Trip: domain=1 delta=49999
Round Trip: domain=2 delta=50000
Round Trip: domain=0 delta=50000
Round Trip: domain=1 delta=50000
Round Trip: domain=2 delta=50000
Round Trip: domain=0 delta=50000
Round Trip: domain=1 delta=50000
Round Trip: domain=2 delta=50000
Round Trip: domain=0 delta=50000
Round Trip: domain=1 delta=50000
Sending to task, DomainTick: 4
Round Trip: domain=2 delta=50000
Round Trip: domain=0 delta=50000
Message received from task, DomainTick: 2
...
```

Which shows that each domain runs in the same number of cycles with a variance of 1. This variance seems negligible, and could most likely be attributed to some small discrepancy in how QEMU handles the machine timer interrupt.

6.2 Larger Demo

A more complex example was also developed to show the capability of creating tasks within each newly generated domain. The initial setup of the demo can be seen in Figure 5. That is, domain 0 creates 4 new domains each with a single privileged

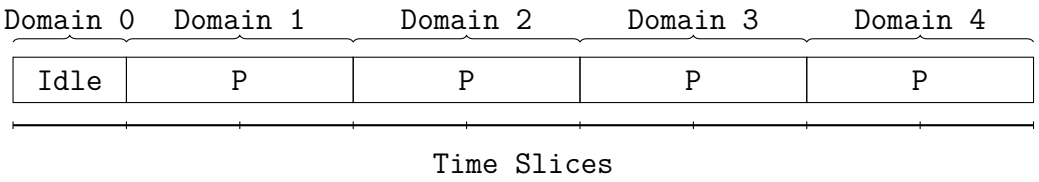


Figure 5: Initial domain full setup. P denotes that it is created with a single privileged task within.

task. Once domain 0 has created these tasks, it starts the scheduler and idles. Each domain runs the same privileged task, which can be seen below:

```
static void domainPrivileged(void* pvParameters) {
    uint32_t dom_num = (uint32_t) pvParameters;
    TickType_t domain_tick = xTaskGetDomainTick();
    printf ( "Privileged task: %d, DomainTick: %d\n", dom_num, domain_tick);
    static StackType_t xPrivilegedUnStack[ NUM_DOMAINS ][
    ↪ configMINIMAL_STACK_SIZE ] __attribute__((aligned(
    ↪ configMINIMAL_STACK_SIZE * sizeof(StackType_t) )));

    TaskParameters_t xDomainTaskParameters =
    {
        .pvTaskCode     = domainUnPrivileged,
        .pcName         = "Rx",
        .usStackDepth   = configMINIMAL_STACK_SIZE,
        .pvParameters   = (void* ) dom_num,
        .uxPriority     = mainUNPRIVILEGED_PRIORITY,
        .puxStackBuffer = xPrivilegedUnStack[dom_num - 1],
        .xRegions       =
        {
            { &_text, _erodata - _text, portMPU_REGION_READ |
    ↪ portMPU_REGION_EXECUTE },
            { &_data, _ebss - _data, portMPU_REGION_READ |
    ↪ portMPU_REGION_WRITE },
            /* NS16550 */
            { (void *)0x10000000UL, 0x1000, portMPU_REGION_READ |
    ↪ portMPU_REGION_WRITE },
            { 0, 0, 0 },
            { 0, 0, 0 },
            { 0, 0, 0 },
            { 0, 0, 0 },
            /* *MUST* reserve one entry for stack */
            { 0, 0, 0 },
```

```
        }
        #if ( configENABLE_DOMAINS == 1 )
        ,.pxDomainParameters  = NULL,
        #endif
    };

    TaskHandle_t xDomTask;

    BaseType_t ret = pdFAIL;
    while (ret != pdPASS) {
        ret = xTaskCreateRestricted( &( xDomainTaskParameters ), &xDomTask
    ↪ );
    }

    // This is never reached, Privileged has lower priority than
    ↪ UnPrivileged
    for( ; ; )
    {
    }
}
```

Each of the privileged tasks start by printing their domain number and the current domain tick. Once that is done, they create a new unprivileged task with a higher priority than themselves. This then immediately schedules the new task, and the privileged task is never scheduled again. The unprivileged task that is created is defined as follows:

```
static void domainUnPrivileged(void* pvParameters) {
    uint32_t dom_num = (uint32_t) pvParameters;
    TickType_t domain_tick = xTaskGetDomainTick();
    printf ( "UnPrivileged task: %d, DomainTick: %d\n", dom_num,
    ↪ domain_tick);

    TickType_t prev_tick = xTaskGetTickCount();
    TickType_t cur_tick = xTaskGetTickCount();

    for( ; ; )
    {
        cur_tick = xTaskGetTickCount();
        TickType_t domain_tick = xTaskGetDomainTick();
        if (prev_tick != cur_tick) {
            printf ( "Domain %d, Domain Tick %d\n", dom_num, domain_tick);
            prev_tick = cur_tick;
        }
```

```
17      }
18  }
```

This unprivileged task again starts by printing its domain and the domain tick, before going into an infinite loop. Within this infinite loop, it checks if a new tick occurred, and prints its own domain along with the domain tick. The overall demo is quite simple, but shows the functionality of creating a new task within a created domain, and one would expect that the provided domain ticks are constant for each domain. Running this demo with the '-icount' QEMU flag gives the following output:

```
Privileged task: 1, DomainTick: 1
UnPrivileged task: 1, DomainTick: 1
Domain 1, Domain Tick 2
Privileged task: 2, DomainTick: 3
UnPrivileged task: 2, DomainTick: 3
Domain 2, Domain Tick 4
Privileged task: 3, DomainTick: 5
UnPrivileged task: 3, DomainTick: 5
Domain 3, Domain Tick 6
Privileged task: 4, DomainTick: 7
UnPrivileged task: 4, DomainTick: 7
Domain 4, Domain Tick 8
Domain 1, Domain Tick 1
Domain 1, Domain Tick 2
Domain 2, Domain Tick 3
Domain 2, Domain Tick 4
Domain 3, Domain Tick 5
Domain 3, Domain Tick 6
Domain 4, Domain Tick 7
Domain 4, Domain Tick 8
```

First, we see that the privileged task from domain 1 runs at the expected domain tick 1. It then creates the unprivileged task within the same tick, and because the unprivileged task has a higher priority than the privileged task the priority based scheduler within domain 1 switches out the privileged task with the unprivileged task all within the same tick. The unprivileged task reaches the infinite loop, and prints that the domain tick incremented to two, before looping until its time slice is up. The same sequence follows for the other domains, before all the unprivileged tasks are the only ones producing output, where we can see the two time slices given to

each of the domains and how the scheduler is upholding the number of time slices for each domain.

6.3 Architectural Complexity and Cost Analysis

While the evaluation demonstrates that the domain-level scheduler successfully achieves functional time slice isolation, this security guarantee introduces substantial structural complexity to the FreeRTOS kernel. The domain-level abstraction diverges significantly from native FreeRTOS scheduling, which alters the behaviour of some of the timing-dependent components.

A consequence of this complexity is the change of standard API assumptions. Take for example the `vTaskDelayUntil` function from FreeRTOS. This function takes the number of ticks the current task should wait until it can be scheduled again. This works via the global `xTicks`, which is now domain specific in the domain scheduler. This means that if a task attempts to sleep for 2 ticks, those two ticks are only counted when the task's own domain is running.

Beyond structural complexity, achieving time protection reduces the overall throughput of tasks by adding idle time to guarantee the time slice scheduling. While it has not been evaluated here, this will almost certainly reduce the average case throughput. Further, the implementation itself adds significant complexity to the code itself, which could be seen as a challenge for future development, where FreeRTOS is generally seen as a quite minimal RTOS implementation, this domain-level scheduler most definitely breaks this assumption.

7 Future work and discussion

While the implementation does show promise, there are still significant items that are left for future work. There are both questions about the verification on true hardware, and a question about the broader scope of safe system calls. Further, while the domain scheduler removes a lot of microarchitectural side channels, through the use of temporal_fence, and removes scheduler based timing channels these are far from all possible timing channels. This section attempts to address these issues, and how future work can overcome these issues.

7.1 Verification on cycle-accurate hardware

As mentioned previously, the QEMU virtual machine does not provide cycle accurate emulation. The closest provided is the instruction-based time counting as described

in [25]. This means that any timing channel that arises from the microarchitectural shared state cannot be emulated or diagnosed on the QEMU platform. What can be empirically evaluated on the platform is a constant time execution path as shown with the previous domain-level scheduler.

During the evaluation phase of this thesis a QEMU TCG plugin was developed in an attempt to bridge this gap. The plugin provided a cache emulation of the L1 and L2 cache with support for flushing the L1 cache on domain switches, and it functioned correctly in emulation. However, QEMU was never designed to produce cycle-accurate emulation, and when the plugin attempted to update the machine timer externally, this happened asynchronously and was affected by the host machine running QEMU. This meant that even though the cache emulation seemed correct, there would be a non-deterministic execution from the guest OS point of view, making time control unusable. While significant work went into the plugin, it ultimately proved unable to provide the cycle-accurate emulation necessary for timing channel analysis. To properly evaluate the implementation, a hardware platform that supports the temporal fence instruction would be ideal. As an alternative, cycle-accurate simulators such as gem5 [26] could prove valuable for future evaluation. While other research has identified some challenges with gem5 [27], such a simulator could significantly reduce the barrier to entry and lead to more rapid development in the pursuit of time protection.

7.2 Safeguarding system calls

With the current setup, when a new domain is created, it must be created with a task that is in privileged mode, if that domain should have more than a single task running. The reason for this is twofold. First, to create a task in FreeRTOS-MPU one must be in privileged mode. Secondly, to add a task to a domain with the new API presented in Section 5.1, one must either be inside the domain itself or be creating a new domain[4]. The requirement of a single privileged task mainly comes from the FreeRTOS-MPU version, and to circumvent this issue would require significant changes from FreeRTOS itself. The larger issue in regard to domain creation is that creating a new domain could delay scheduling. Specifically, many of the privileged functions enter a ”critical” section within FreeRTOS. When entering such a critical section, on the RISC-V MPU version interrupts are disabled, including the machine timer interrupt, which drives the domain scheduling. This poses a significant burden on the privileged tasks within each domain, as they can inadvertently create timing

[4]Given the task runs in privileged mode, it could technically overwrite the memory locations forcefully, and not make use of the api at all.

side-channels. The issue is not only with the domain creation, but any call to functions that enter the critical sections.

This issue poses a few challenges, and there are two primary solutions, which could be considered for future development. The first is taking inspiration from [9]. Here, they clone the shared kernel data into memory provided by user level tasks within each domain. The current implementation created within FreeRTOS does not do this, but many of the preexisting features of the within domain priority scheduler could be fully partitioned. For this to function, one would have to introduce two different kinds of critical sections. There is the cross-domain critical section, for which it would still be necessary to disable interrupts when it occurs, and there is the within domain critical section. When entering a within domain critical section, then the priority scheduling would be blocked such that the priority based scheduler is not left in an invalid state, but if there is a domain-level interrupt, then the context would be saved of the current execution and switch to another domain. When the old domain runs again, it would still be within the domain critical section at the same context point, allowing for a valid state of the within domain scheduler, while allowing the domain-level scheduler to schedule other domains without delays. This context saving and restoration would have to be taken into account for the worst case analysis of the domain switching, if it can not be guaranteed in constant time, but should propagate well to the majority of FreeRTOS' standard implementation.

The next issue would be the cross-domain critical section. This part of the kernel is shared between all, and as such one cannot simply "ignore" the critical section as before. Instead one would have to guarantee that any call to cross domain critical functions would execute within the current domain. A solution to this problem would be to guarantee that any system call with such a cross-domain critical section runs within a single time slot, or allows for preemption within a time slice. Then, when entering the cross-domain critical section, the task would have to idle until it reaches the start of a time slice and then execute the system call. This would mean that execution is guaranteed to finish before the scheduler would attempt to schedule the next domain.

For this to function correctly one would need proper hardware to find the running time of all such system calls, and make sure that the execution time is bounded by the time slice. It would also mean that there has to be a hard requirement on the lowest possible value of the time slice duration. However, with these in mind truly isolated domains should be possible.

Note however, that this side-channel would only be possible during the end of a domain's time slice, and would only be able to convey information between the current domain and the domain which comes immediately after. As an example, I

believe this could be a possible timing channel within the S3K kernel. S3K makes use of a global TTAS lock. A lock is acquired through the following code snippet:

```
bool ttas_acquire(ttas_t *ttas, bool preemptable)
{
  while (__atomic_exchange_n(&ttas->lock, 1, __ATOMIC_ACQUIRE)) {
    if (preemptable && preempt()) {
      return false;
    }
  }
  return true;
}
```

The preemptable flag is used to indicate whether the current lock acquisition will respect a preemption. The preempt function call checks if the mip register has a bit set at (1 « 7), which corresponds to the Machine Timer Interrupt Pending (MTIP) bit [28]. This bit indicates that the machine timer is pending and should fire. In S3K this lock is acquired in two important places, namely:

```
proc_t *syscall_handler(void)
{
  // The system call number.
  word_t syscall_nr = current->regs.a0;

  // If system call number is invalid, make an exception.
  if (syscall_nr >= ARRAY_SIZE(handlers)) {
    return exception_handler(0x8, syscall_nr);
  }

  // Try to acquire a lock. Also checks for preemption.
  if (!lock_acquire(true)) {
    return NULL;
  }

  // Advance the program counter.
  current->regs.pc += 4;

  // Call the system call handler
  proc_t *next = handlers[syscall_nr](current->pid, &current->regs.a0);

  // Releases the lock.
  lock_release();

  return next;
```

```
}

static proc_t *sched_next(hart_t hart, uint64_t *timeout)
{
  bool swapped = false;

  // Lock because other processes may be accessing the schedule
  lock_acquire(false);
  uint64_t rtc_slot = sched_rtc_slot();
  uint64_t offset = curr[hart] % MAX_TIME_SLOT;
  // Advance curr if the current slot has expired
  if (curr[hart] + schedule[hart][offset].length <= rtc_slot) {
    curr[hart] += schedule[hart][offset].length;
    swapped = true;
  }
  frame_t slot = schedule[hart][curr[hart] % MAX_TIME_SLOT];
  *timeout = slot2time(curr[hart] + slot.length);
  lock_release();
  // Release lock because we do not want to block when executing temporal
    ↪ fence.

  if (swapped) {
    temporal_fence(); // Insert a temporal fence if we swapped slots
  }

  if (slot.pid == INVALID_PID) {
    return NULL; // No process scheduled for this slot
  }

  // We now have a process we may schedule.
  proc_t *proc = proc_get(slot.pid);
  if (proc->timeout > slot2time(rtc_slot)) {
    return NULL; // Process is sleeping or waiting
  }

  // Try to acquire the process
  lock_acquire(false);
  bool proc_acquired = proc_acquire(slot.pid);
  proc->timeout = *timeout;
  lock_release();
  return proc_acquired ? proc : NULL;
}
```

That is, when a system call is made the TTAS lock is acquired, and when a new task is scheduled the same lock is also acquired. Now while the preemptable flag is

true for the syscall, it checks that no preemption is pending while gathering the lock, but once it has the lock, it will delay scheduling if the scheduler runs while the syscall holds the lock. The syscalls in S3K attempt to rectify this by checking for preemption after a constant amount of work so as not to block the scheduler for too long, but if this constant time varies one these syscalls could be used in unintended ways and a domain could in theory cause predictable scheduling delay and relay information to the next domain. This analysis is purely theoretical and would require access to hardware to fully test, and as such should mainly be seen as one of the multitude of challenges a truly time protected kernel will face.

7.3 The FreeRTOS kernel is vast

One requirement for time protection specified within [9] is that the OS must be partitioned. This includes all system calls, which must be constant time in regard to available information within each domain. Given that the domain scheduler implemented creates a copy of the priority based scheduler for each domain, it can be assumed that a large portion of these system calls indirectly become constant time in this aspect. Further, as the User mode tasks are unable to disable any interrupts this would indicate that the shared front that remains to be partitioned is anything regarding the domain-level scheduler itself. Given that these do make use of constant-time operations, and domain creation is restricted to Privileged mode, then it would seem that this requirement might be met to some extent. However, to truly show this partitioned OS one would need a full audit of all system calls available for User mode tasks, and make sure that they are constant time with respect to the domain available information. As this is something beyond the scope of this thesis, the implementation can provide no guarantees that it is truly implementing time protection.

7.4 Remaining timing channels

While scheduling timing channels are a significant vulnerability, they are far from the only timing channel present in modern kernels. As such, the provided port of the FreeRTOS kernel is by no means time protected. One significant form of timing channel still available within the kernel is that of interrupts. During development it was assumed that the only interrupt is that of the preemption timer used for scheduling. In a real system this is not the case. Here, one would have to handle the external interrupts in a deterministic manner, without breaking the scheduling guarantees put forth during the time slot based scheduling. It might also be desired

that some domains are not interrupted during execution, as this gives information about the availability of some external service that in itself might be deemed an information leak. It is up to future work to determine a method for handling these external interrupts in a manner where they uphold both of the aforementioned issues.

7.5 Partitioned hardware

While this thesis makes use of the `fence.t` instruction to clear the microarchitectural state on a domain switch, a lot of the hardware still remains shared. Larger caches such as the L2 and L3 cache are too large to simply flush [17]. This means higher level caches need another method for partitioning. In [9] they make use of cache coloring to partition the higher level caches. In essence cache coloring works by using some of the bits in the physical address to encode a color of the page. For example, if using 2 bits in the physical address, one can encode 4 colors, and when the application asks to allocate consecutive pages, then the OS makes sure to map them to the correct colors [29], [30]. The L1 cache is usually indexed directly with virtual addresses, and as such can not be colored in this manner, whereas higher-level caches more often make use of elements of the physical address.

Cache coloring utilises the cache in an unintended manner. In general, it reduces the average performance of the entire system, while for some specific applications it can improve performance by reducing cache interference. However, in truly secure kernel software partitioning is still required and as such it is a cost that is inevitable. The larger issue with cache coloring is the uncertainty that it poses. Where the Instruction Set Architecture(ISA) usually standardizes the architectural interface for virtual memory and the MMU behaviour, the same cannot be said for the underlying foundation of cache coloring. For example, in RISC-V the ISA specifies the programmer-visible MMU model, while the underlying cache implementation remains undefined. This means that the cache coloring implementation is CPU-specific based on things like cache indexing. This makes a cross platform implementation of cache coloring challenging, and any future development in cache architecture could break any such implementation [30].

In recent years different architectures have added support for hardware cache partitioning. For example, Intel have added CAT [31], ARM with the MPAM extension [32] and similar work for RISC-V with the QoS register interfaces [33]. For a truly secure kernel, these items need to reach a maturity level, where domains can be guaranteed separate in the higher level caches that previously could not be partitioned.

Another approach for safe OS execution is what was seen in [8]. Here they make

use of the Scratch Pad Memory (SPM) which is a section of memory that is never backed up to the higher level caches. The kernel itself was then developed, such that all memory accesses by the kernel only left a footprint in the SPM, which was then flushed on a domain switch. This approach seems promising for the kernel data itself, assuming that the SPM is guaranteed large enough to hold the kernel footprint. Even when cache partitioning fully matures, this might still be a part of the solution, as it most likely would have performance improvements over cache partitioning, and would reduce the interference the OS would have on the user level applications using the higher level caches.

Most likely there will never be a hardware solution which fits all. A general purpose operating system, where user-level constant-time programming can solve the most important issues, might not need specific domain separation and cache partitioning. On the other hand, in high security environments this is a must and something where depending on the use case the correct solution could vary.

8 Conclusion

Timing channels pose a security threat in modern operating systems. As more of the simple security issues get patched, more advanced methods of breaking OS barriers become prevalent. Prior work exists in creating secure microkernels [8], [9] as well as promising developments in the user-level mitigations [6]. However, there still exists a gap in how time protection can be implemented in existing operating systems.

To bridge this gap, this work proposes a domain-level scheduler paired with flushing during domain switches. Conceptually, this implementation isolates domains by partitioning execution time into a fixed number of time slices, which are then dynamically distributed among them. When a domain switch occurs, the microarchitectural footprint of the previous domain is cleared via the temporal fence instruction [17], [18], preventing cross-domain information leakage through the shared microarchitectural state.

To realize this model, the implementation is built as a modification of the FreeRTOS RISC-V MPU port, such that MPU protections can be used to completely separate domains. Because FreeRTOS natively uses a priority-based scheduler, it is inherently vulnerable to scheduler-based timing attacks where malicious tasks can infer information from scheduling behaviour. To resolve this, the domain-level scheduler is introduced on top to manage time slices, while preserving the native priority-based scheduler within each individual domain. This approach creates a middle ground, where one can optimize throughput when time protection is unnecessary. To allow for an intended communication channel across domain boundaries, the preexisting

Queue structure was modified to allow for cross-domain communication.

Empirical evaluation shows that the implementation is functional. Within the QEMU-based system emulation, the framework achieves strictly constant-time time slices, with resulting round-trip times showing a stable cycle count of 50,000, with a variance of just a single cycle.

However, the implementation is not without its limitations. Notably, it changes some of the fundamental expectations in FreeRTOS and adds a significant amount of complexity to the scheduler. There is also known shared state still present, such as the higher-level caches, which is not isolated through temporal or spatial partitioning. Some of these areas require more help from hardware, which would allow for either spatial partitioning or temporal partitioning, such as in the case of the scratchpad memory used in the S3K kernel [8]. Further, the evaluation itself was significantly limited by the lack of real-world hardware. The evaluation does not take into account the underlying microarchitectural state, and as such, the implementation can only theorize on the efficacy of the temporal fence instruction on domain switches. It remains as future work to test the implementation on hardware where such an instruction is present.

Even with this, this thesis has contributed by showing that time protection can be added into an existing mainstream kernel, while showing intended behaviour on the QEMU system emulation platform. It shows that time protection is achievable, but comes with significant trade-offs in complexity, performance and usability. So while a full domain-level scheduler is most likely not suitable for more generic operating systems, it remains viable for environments where a specialized microkernel with security is needed.

With this in mind, there are still aspects of time protection that could be used in the more generic operating systems. Specifically, future research can make use of these concepts:

- **Selective Temporal Partitioning:** Given the low performance overhead of the temporal fence instruction (1% on average), it could be used in a generic operating system to create sensitive tasks. The operating system could flush the microarchitectural state before and after these sensitive tasks are scheduled. This would eliminate their microarchitectural footprint while allowing for more freedom in execution for the sensitive tasks.

- **Hardware-Assisted partitioning:** In conjunction to this, hardware features such as cache partitioning and scratchpad memory could guarantee that non-flushable components also remain partitioned from these specific sensitive tasks.

- **User level cooperation:** With safe hardware, the user level mitigations could cover the remaining attack front in these general purpose environments, and could lead to a safe execution environment for known sensitive tasks.

Finally, to fully guarantee the security contract involved in time protection, significant work remains in formal verification. As far as this thesis is concerned, this remains an open challenge.

References

[1] J.-F. Dhem, F. Koeune, P.-A. Leroux, P. Mestré, J.-J. Quisquater, and J.-L. Willems, "A Practical Implementation of the Timing Attack," in *Smart Card Research and Applications*, J.-J. Quisquater and B. Schneier, Eds., red. by G. Goos, J. Hartmanis, and J. Van Leeuwen, vol. 1820, Berlin, Heidelberg: Springer Berlin Heidelberg, 2000, pp. 167–182, ISBN: 978-3-540-67923-3 978-3-540-44534-0. DOI: 10.1007/10721064_15 Accessed: Feb. 17, 2026. [Online]. Available: http://link.springer.com/10.1007/10721064_15

[2] O. Aciiçmez, W. Schindler, and Ç. K. Koç, "Improving Brumley and Boneh timing attack on unprotected SSL implementations," in *Proceedings of the 12th ACM Conference on Computer and Communications Security*, ser. CCS '05, New York, NY, USA: Association for Computing Machinery, Nov. 7, 2005, pp. 139–146, ISBN: 978-1-59593-226-6. DOI: 10.1145/1102120.1102140 Accessed: Feb. 17, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/1102120.1102140

[3] T. Ristenpart, E. Tromer, H. Shacham, and S. Savage, "Hey, you, get off of my cloud: Exploring information leakage in third-party compute clouds," in *Proceedings of the 16th ACM Conference on Computer and Communications Security*, Chicago Illinois USA: ACM, Nov. 9, 2009, pp. 199–212, ISBN: 978-1-60558-894-0. DOI: 10.1145/1653662.1653687 Accessed: Feb. 4, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/1653662.1653687

[4] M. Lipp et al., "Meltdown: Reading kernel memory from user space," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018.

[5] J. Corbet, "KAISER: Hiding the kernel from user space," *LWN.net*, Nov. 15, 2017. Accessed: Feb. 17, 2026. [Online]. Available: https://lwn.net/Articles/738975/

[6] S. Cauligi et al., "FaCT: A DSL for timing-sensitive computation," in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2019, New York, NY, USA: Association for Computing Machinery, Jun. 8, 2019, pp. 174–189, ISBN: 978-1-4503-6712-7. DOI: 10.1145/3314221.3314605 Accessed: Feb. 3, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3314221.3314605

[7] P. Kocher et al., "Spectre attacks: Exploiting speculative execution," in *40th IEEE Symposium on Security and Privacy (S&P'19)*, 2019.

[8] H. Karlsson and R. Guanciale, "Partitioning Kernel With Capability Controlled Temporal and Spatial Partitioning," in *2025 IEEE Real-Time Systems Symposium (RTSS)*, Dec. 2025, pp. 68–81. DOI: 10.1109/RTSS66672.2025.00015 Accessed: Feb. 4, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11315078

[9] Q. Ge, Y. Yarom, T. Chothia, and G. Heiser, "Time Protection: The Missing OS Abstraction," in *Proceedings of the Fourteenth EuroSys Conference 2019*, ser. EuroSys '19, New York, NY, USA: Association for Computing Machinery, Mar. 25, 2019, pp. 1–17, ISBN: 978-1-4503-6281-8. DOI: 10.1145/3302424.3303976 Accessed: Feb. 3, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3302424.3303976

[10] T. Murray et al., "seL4: From General Purpose to a Proof of Information Flow Enforcement," in *2013 IEEE Symposium on Security and Privacy*, Berkeley, CA: IEEE, May 2013, pp. 415–429, ISBN: 978-0-7695-4977-4 978-1-4673-6166-8. DOI: 10.1109/SP.2013.35 Accessed: Feb. 17, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/6547124/

[11] S. Buckley et al. "Proving the Absence of Microarchitectural Timing Channels." version 1. arXiv: 2310.17046 [cs], Accessed: Feb. 10, 2026. [Online]. Available: http://arxiv.org/abs/2310.17046 pre-published.

[12] M. Schneider, D. Lain, I. Puddu, N. Dutly, and S. Capkun, "Breaking Bad: How Compilers Break Constant-Time Implementations," in *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, Aug. 25, 2025, pp. 1690–1706. DOI: 10.1145/3708821.3733909 arXiv: 2410.13489 [cs]. Accessed: Feb. 17, 2026. [Online]. Available: http://arxiv.org/abs/2410.13489

[13] D. A. Osvik, A. Shamir, and E. Tromer, "Cache Attacks and Countermeasures: The Case of AES," in *Topics in Cryptology – CT-RSA 2006*, D. Pointcheval, Ed., red. by D. Hutchison et al., vol. 3860, Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 1–20, ISBN: 978-3-540-31033-4 978-3-540-32648-9. DOI: 10.1007/11605805_1 Accessed: Feb. 10, 2026. [Online]. Available: http://link.springer.com/10.1007/11605805_1

[14] F. Liu, Y. Yarom, Q. Ge, G. Heiser, and R. B. Lee, "Last-Level Cache Side-Channel Attacks are Practical," in *2015 IEEE Symposium on Security and Privacy*, May 2015, pp. 605–622. DOI: 10.1109/SP.2015.43 Accessed: Feb. 4, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/7163050

[15] C.-Y. Chen, S. Mohan, R. Pellizzoni, R. B. Bobba, and N. Kiyavash, "A Novel Side-Channel in Real-Time Schedulers," in *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, Apr. 2019, pp. 90–102. DOI: 10.1109/RTAS.2019.00016 Accessed: Feb. 9, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/8743170/

[16] S. Kadloor, N. Kiyavash, and P. Venkitasubramaniam, "Mitigating Timing Side Channel in Shared Schedulers," *IEEE/ACM Transactions on Networking*, vol. 24, no. 3, pp. 1562–1573, Jun. 2016, ISSN: 1558-2566. DOI: 10.1109/TNET.2015.2418194 Accessed: Feb. 9, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/7089318/

[17] N. Wistoff, M. Schneider, F. K. Gürkaynak, G. Heiser, and L. Benini, "Systematic prevention of on-core timing channels by full temporal partitioning," *IEEE Transactions on Computers*, vol. 72, no. 5, pp. 1420–1430, May 2023, ISSN: 2326-3814. DOI: 10.1109/tc.2022.3212636 [Online]. Available: http://dx.doi.org/10.1109/TC.2022.3212636

[18] N. Wistoff, G. Heiser, and L. Benini, "Fence.t.s: Closing Timing Channels in High-Performance Out-of-Order Cores Through ISA-Supported Temporal Partitioning," in *Applications in Electronics Pervading Industry, Environment and Society*, M. Ruo Roch, F. Bellotti, R. Berta, M. Martina, and P. Motto Ros, Eds., Cham: Springer Nature Switzerland, 2025, pp. 269–276, ISBN: 978-3-031-84100-2. DOI: 10.1007/978-3-031-84100-2_32

[19] J. Rushby, "Avionics Architectures: Mechanisms, and Assurance," 1999.

[20] "What is FreeRTOS? - FreeRTOS™," Accessed: Apr. 7, 2026. [Online]. Available: https://freertos.org/Why-FreeRTOS/What-is-FreeRTOS

[21] "Memory Protection Unit (MPU) Support - FreeRTOS™," Accessed: Apr. 7, 2026. [Online]. Available: https://freertos.org/Security/04-FreeRTOS-MPU-memory-protection-unit

[22] "FreeRTOS-Community-Supported-Demos/RISC-V_RV32_MPU_QEMU_VIRT_GCC at main · FreeRTOS/FreeRTOS-Community-Supported-Demos," GitHub, Accessed: Apr. 7, 2026. [Online]. Available: https://github.com/FreeRTOS/FreeRTOS-Community-Supported-Demos/tree/main/RISC-V_RV32_MPU_QEMU_VIRT_GCC

[23] R. L. Schröder, S. Gast, and Q. Guo, "Divide and Surrender: Exploiting Variable Division Instruction Timing in HQC Key Recovery Attacks,"

[24] Y. Jin, P. Qiu, C. Wang, Y. Yang, D. Wang, and G. Qu. "Timing the Transient Execution: A New Side-Channel Attack on Intel CPUs." arXiv: 2304.10877 [cs], Accessed: Apr. 29, 2026. [Online]. Available: http://arxiv.org/abs/2304.10877 pre-published.

[25] "TCG Instruction Counting — QEMU documentation," Accessed: Apr. 28, 2026. [Online]. Available: https://www.qemu.org/docs/master/devel/tcg-icount.html

[26] J. Lowe-Power et al. "The gem5 Simulator: Version 20.0+." arXiv: 2007.03152 [cs.AR], Accessed: May 14, 2026. [Online]. Available: http://arxiv.org/abs/2007.03152 pre-published.

[27] L. Bossuet, V. Grosso, and C. A. Lara-Nino, "Emulating Side Channel Attacks on gem5: Lessons learned," in *2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, Delft, Netherlands: IEEE, Jul. 2023, pp. 287–295, ISBN: 979-8-3503-2720-5. DOI: 10.1109/EuroSPW59978.2023.00036 Accessed: Mar. 23, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/10190725/

[28] "Mip :: RISC-V ISA Manual," Accessed: May 14, 2026. [Online]. Available: https://riscv.github.io/riscv-unified-db/manual/html/isa/isa_20240411/csrs/mip.html#udb:doc:csr_field:mip:MTIP

[29] A. Dinh. "Cache Coloring," Accessed: May 13, 2026. [Online]. Available: https://dinhtta.github.io/cache/

[30] "What is Cache Coloring and How Does it Work?" Accessed: May 13, 2026. [Online]. Available: https://www.lynx.com/blog/what-is-cache-coloring

[31] "Introduction to Cache Allocation Technology in the Intel® Xeon®...," Intel, Accessed: May 13, 2026. [Online]. Available: https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-cache-allocation-technology.html

[32] "Learn the architecture - Memory System Resource Partitioning and Monitoring (MPAM) Overview," Accessed: May 13, 2026. [Online]. Available: https://developer.arm.com/documentation/107768/0101/Arm-Memory-System-Resource-Partitioning-and-Monitoring--MPAM--Extension

[33] "1.1. Introduction :: RISC-V Ratified Specifications Library," Accessed: May 13, 2026. [Online]. Available: https://docs.riscv.org/reference/cbqri/qos_intro.html

A Getting the source code

The code for this thesis is hosted on github at `https://github.com/svbrodersen/Speciale`. This repository contains the work done for the thesis including the report itself, and the different submodule projects used throughout the thesis. Further, to reproduce the results a podman image is provided to get a container capable of building and running the demo examples. The container is hosted at `ghcr.io/svbrodersen/speciale:latest`. To help with the setup, the root of the repository contains a README.md file with instructions on how to build and run the demos along with how to setup the container. To clone the repository run the following commands:

```
git clone https://github.com/svbrodersen/Speciale.git
git submodule update --init --recursive
```

As the project has quite some large submodules, beware that this might take up a significant amount of space, and could take some time.

The primary source code for the modified FreeRTOS kernel is hosted at: `https://github.com/svbrodersen/FreeRTOS-Kernel/tree/main`, and it should be pulled automatically following the submodule recursive update.